

课程讲义

CS231N Spring 2025 课程讲义: Lecture 15 3D Vision

Codex Worker 3

2026-04-15



视频作者/频道: Stanford Online

发布日期: 2025-09-02

视频时长: 1:11:50

视频链接: <https://www.youtube.com/watch?v=7lxrKDKtykM>

目录

1	三维表示：从点、网格到参数曲面	2
1.1	本章小结	2
2	显式与隐式表示：表达能力、操作性与计算代价	3
2.1	本章小结	3
3	从手工表示到可学习表示：3D 数据上的深度模型	3
3.1	本章小结	4
4	神经隐式表示、NeRF 与高效渲染	4
4.1	本章小结	5
5	总结与延伸	5

1 三维表示：从点、网格到参数曲面

这讲的主线非常鲜明：三维视觉首先不是“某个网络结构”，而是“你打算用什么表示来承载三维几何”。讲者一开始就把问题拆开：三维表示必须能被计算机存储、能支持编辑与操作、能适应不同输入来源，并且在渲染、重建、分析时具有合适的计算特性。换言之，表示本身就是方法设计的一部分。

最基础的表示是点和点云（point clouds）。点云把对象写成一组无序的三元组，优点是结构简单、几乎所有深度传感器和扫描系统都能直接给出这样的原始输出；缺点是缺乏连通关系，因此很难直接描述平滑表面、局部拓扑或 inside/outside 结构。

Point Clouds



- Easily represent any kind of geometry
- Useful for large datasets
- Difficult to draw in undersampled regions
- Other limitations:
 - No simplification or subdivision
 - No direction smooth rendering
 - No topological information

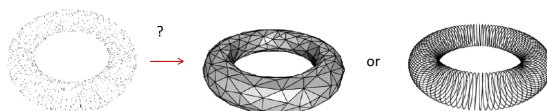


图 1：点云是最直接的三维表示之一：结构简单、来源广泛，但缺乏连通性与表面语义。

与点云相比，网格（polygon meshes）把几何表示为顶点与面片的边界表示（boundary representation）。它更接近传统图形学和 CAD 里的对象表达，因此非常适合显示、编辑和局部几何处理。课程用 Digital Michelangelo 等超大网格案例提醒学生：网格可以非常精细，但也会带来顶点数、采样密度和后处理复杂度上的压力。

进一步，参数表示（parametric representation）则试图用连续函数来定义曲线与曲面。例如二维参数曲面可写成：

$$f(u, v) = (x(u, v), y(u, v), z(u, v)).$$

其中 u, v 是参数空间中的坐标， f 把低维参数域映射到三维空间。当映射平滑且可控时，生成的曲面也会天然平滑。课程用 Bézier curves / surfaces 说明了参数表示的优势：如果你有良好的控制点和组合规则，就能用相对少的参数得到非常平滑的几何体。

1.1 本章小结

三维表示的第一层分歧，在于你是更关心原始采样便利性，还是更关心表面结构、编辑能力与平滑性。点云、网格、参数曲面分别代表了这几种取向，没有谁在所有任务里天然占优。

2 显式与隐式表示：表达能力、操作性与计算代价

当课程从显式（explicit）表示转向隐式（implicit）表示时，真正的主题变成了：同一对象，是否可以通过“满足某个函数条件的点集合”来定义？隐式表示常写成

$$f(x, y, z) = 0,$$

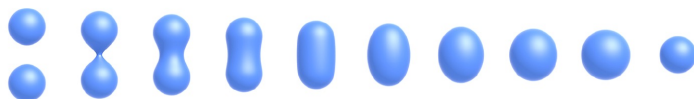
也就是说，所有满足某个约束的点构成目标曲面。若使用 signed distance function (SDF) 等距离函数，则函数值还可以告诉我们点到表面的有符号距离，因此 inside/outside test 变得极其自然。

Distance Functions (Implicit)

Instead of Boolean, gradually blend surfaces together using

Distance functions:

Giving minimum distance (could be **signed** distance) from anywhere to object



Slide credit: Ren Ng

图 2: 距离函数等隐式表示擅长 inside/outside 判断与布尔操作，但采样和可视化并不总是直接。

这带来一组很漂亮的对照：显式曲面容易直接采样和渲染，但做内外测试、布尔组合或拓扑操作可能更麻烦；隐式曲面很适合回答“这个点在内部还是外部”，也容易做合并、切割等运算，但把它转换成可视表面往往需要额外算法；voxels 可以看成在离散网格上保存 occupancy 或密度的另一条路线，规则网格让计算方便，但分辨率代价会随三维体积迅速膨胀。

讲者把 level set methods、medical imaging、distance fields 放在一起讲，正是为了说明：隐式表示从来不只是神经网络时代的新发明，而是在几何处理和科学计算里早已有深厚基础。神经隐式方法真正做的，是把这些老观念与学习式参数化结合起来。

2.1 本章小结

显式表示和隐式表示没有绝对高下。前者更接近“你看见的表面”，后者更接近“你定义对象的规则”。不同任务关心的计算原语不同，因此会偏好不同表示。

3 从手工表示到可学习表示：3D 数据上的深度模型

课程中段把视角从“如何表示”推进到“如何学习”。随着 3D datasets 的扩展，研究问题也从简单分类延展到检索、重建、部件理解、室内场景建模和生成。表示选择不再只是几何学问题，而是直接影响模型架构。

例如，多视角卷积网络（Multi-View CNN）把 3D 对象渲染成多个 2D 视角，然后用成熟的 2D CNN 提取特征。这条路线利用了图像网络的成熟生态，但牺牲了部分原生 3D 结构。体素方法（voxels）把 3D 数据规整到栅格中，便于使用三维卷积，但高分辨率成本昂贵。PointNet 则走了完全不同的路线：直接吃点集本身，把“点云是无序集合”视为设计起点。

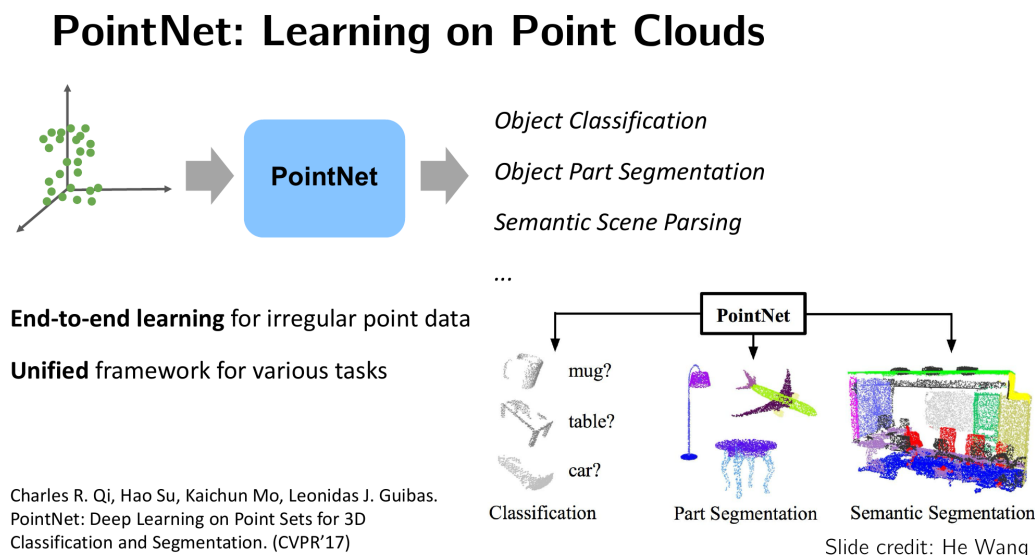


图 3: PointNet 把点云直接作为无序集合输入，用对称聚合实现 permutation invariance。

PointNet 的核心思想是 permutation invariance：如果输入点的顺序被打乱，语义不应改变。为此，模型把每个点先独立编码，再用 max pooling 等对称函数做全局聚合。这一设计看似简单，却极大影响了后续一整条点云网络发展路线。课程用它说明了一条重要方法论：好的表示不仅决定你“存什么”，也决定你“必须尊重什么不变量”。

讲者随后还讨论了图神经网络、距离度量、octree、AtlasNet 等方法，说明 3D 学习领域的一个长期主题是：在离散性、可扩展性和结构表达力之间不断做折中。比如 octree 试图在保留体素规整性的同时提升稀疏区域效率；AtlasNet 则通过多个可学习参数片把二维参数域拼成三维表面，重新回到参数化曲面的思想。

3.1 本章小结

深度模型并没有“替代”表示，而是在每一种表示上重新设计不变量、归纳偏置和计算路径。理解 3D 学习，必须同时理解网络与表示如何互相塑造。

4 神经隐式表示、NeRF 与高效渲染

后半讲最重要的升级，是把“用神经网络表示几何”这件事做到了场景级和渲染级。对于 NeRF (Neural Radiance Fields) 而言，网络接收三维位置与视角方向，输出密度 σ 与颜色或辐射值 c 。于是渲染就不再是传统网格光栅化，而是沿着射线做体渲染 (volume rendering)：

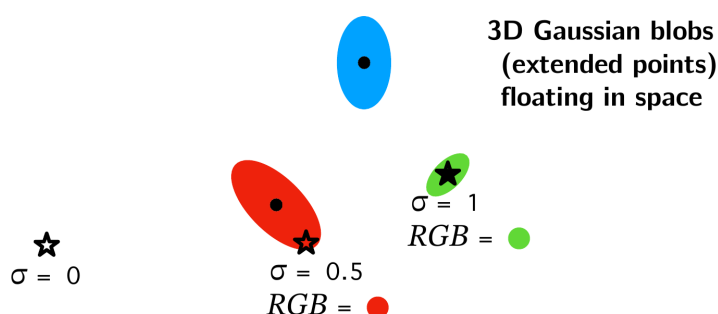
$$C(r) = \int T(t) \sigma(r(t)) c(r(t), d) dt,$$

其中 $r(t)$ 表示相机射线在深度 t 处的位置， $\sigma(r(t))$ 是该点的密度， $c(r(t), d)$ 是给定视角 d 的颜色或辐射，而 $T(t)$ 是透射率，表示前方光线未被遮挡的程度。

课程在 transcript 里把这个过程解释得非常图形学：你在三维空间采样点，拿到颜色与密度，再根据“沿途有多少光被挡住”决定像素最终颜色。其关键突破在于，这一渲染过程可微，因此可以直接用 2D 图像监督来训练 3D 场景函数。

但讲者同样没有把 NeRF 讲成终点。NeRF 的场景表示是密集的，渲染质量高，但代价也高；因此 3D Gaussian Splatting (3DGS) 很快成为新的高效替代。它把场景表示为一组稀疏高斯原语，只在“真正有内容的地方”分配可渲染参数，由此大幅提高渲染速度。

Gaussian splatting parameterizes the scene sparsely, only where density is nonzero.



Slide credit: Vincent Sitzmann

图 4: 3D Gaussian Splatting 通过稀疏高斯原语实现高质量、快速渲染，成为 NeRF 之后的重要表示。

课程给出的比较非常实用：NeRF 与 3DGS 在视觉质量上可以接近，但 3DGS 的效率优势极大，能够把原本慢速的场景渲染推向接近实时的层级。最后几页关于 structure-aware representations、primitives、hierarchical graph 的讨论，则把问题再次提升：仅有稠密场景函数可能还不够，我们还希望表示显式地保留部件、层次与组合结构。

4.1 本章小结

神经隐式表示把几何、外观与渲染统一进一个可学习函数中；NeRF 把这一路线推向高质量场景重建，3DGS 则把效率问题拉回到工程可用区间。两者共同说明：现代 3D 视觉的核心，不只是识别几何，更是学会在表示、渲染和优化之间闭环。

5 总结与延伸

Lecture 15 最大的价值，是把 3D 视觉从“再多加一个维度”矫正为“必须重新思考表示”。点云、网格、参数曲面、隐式函数、体素、NeRF、3DGS，这些看似分散的技术，其实都围绕同一个中心问题展开：我们要如何在有限计算中表达真实世界的几何与外观，并让模型能学、能推理、能渲染、能交互。

如果把本讲当教材来复习，应牢牢记住两层结构。第一层是表示论：不同几何表示服务于不同操作原语，因此任务本身会倒逼表示选择。第二层是学习论：一旦选定表示，网络结构、损失函数、

采样策略和渲染过程都会随之改变。3D 视觉之所以比 2D 视觉复杂，不是因为公式更长，而是因为“世界结构”真的被带回了模型内部。